

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Научный руководитель ОП
«Программная инженерия», профессор
департамента программной
инженерии факультета компьютерных
наук

_____ А. И. Легалов

«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
"Программная инженерия",
старший преподаватель департамента
программной инженерии

_____ Н. А. Павлочев

«__» _____ 2026 г.

**КРОСС-КОМПИЛЯТОР ЯЗЫКА ПРОГРАММИРОВАНИЯ OBERON-7 В
АСЕМБЛЕР ПРОЦЕССОРА RISC-V СИМУЛЯТОРА RARS.**

Руководство оператора

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.04.01 34 01-1-ЛУ

Исполнители:

Студент группы БПИНЖ232

_____ / А. А. Акулов /

«__» _____ 2026 г.

Инва.№ подп	Подп. и дата	Взам. инв.№	Инва.№ дубл.	Подп. и дата

УТВЕРЖДЕН

RU.17701729.05.04.01 34 01-1-ЛУ

**КРОСС-КОМПИЛЯТОР ЯЗЫКА ПРОГРАММИРОВАНИЯ OBERON-7 В
АССЕМБЛЕР ПРОЦЕССОРА RISC-V СИМУЛЯТОРА RARS.**

Руководство оператора

RU.17701729.05.04.01 34 01-1

Листов 13

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. НАЗНАЧЕНИЕ ПРОГРАММЫ	3
1.1. Функциональное назначение	3
1.2. Эксплуатационное назначение	3
2. УСЛОВИЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ	4
2.1. Минимальный состав аппаратных средств	4
2.2. Минимальный состав программных средств	4
2.2.1. Обязательный состав программных средств	4
2.2.2. Опциональный состав программных средств	4
2.3. Требования к персоналу (пользователю)	4
3. Выполнение программы	5
3.1. Установка программы	5
3.1.1. Скачивание программы	5
3.1.2. Сборка программы	5
3.1.3. Запуск программы	6
3.1.4. Примеры использования программы в различных режимах	7
4. СООБЩЕНИЯ ОПЕРАТОРУ	9
4.1. Тексты сообщений оператору	9
5. ТЕСТИРОВАНИЕ	11
5.1. Юнит-тесты	11
5.2. E2E-тесты	11

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. НАЗНАЧЕНИЕ ПРОГРАММЫ

1.1. Функциональное назначение

Функциональное назначение кросс-компилятора o7gc – это преобразование кода языка программирования более высокого уровня (oberon-7), в код языка более низкого уровня, в данном случае в ассемблерный код архитектуры RISC-V, совместимый с эмулятором RARS.

1.2. Эксплуатационное назначение

Продукт предназначен для использования разработчиками программного обеспечения в учебных и исследовательских целях. Помимо процесса преобразования кода для нужной архитектуры, компилятор демонстрирует стандартные возможности современных компиляторов, такие как семантический анализ и встраивание оптимизаций.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. УСЛОВИЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ

2.1. Минимальный состав аппаратных средств

1. Процессор с частотой не менее 1 ГГц.
2. 1 ГБ оперативной памяти.
3. 200 МБ свободного места для приложения.

2.2. Минимальный состав программных средств

2.2.1. Обязательный состав программных средств

1. C++ компилятор соответствующий стандарту C++17.
2. Система сборки cmake 3.20.

2.2.2. Опциональный состав программных средств

1. Flex - генератор лексеров, необходим если включена опция USE_FLEX=ON при сборке.
2. Bison версии 3.0 - генератор парсеров, необходим если включена опция USE_BISON=ON при сборке.
3. Java версии 8, необходима для запуска симулятора RARS.
4. Библиотеки libgc-dev и bdw-gc - необходимы для сборки OBNC, в случае e2e тестирования.
5. RARS - симулятор RISC-V, необходим для e2e тестирования.

2.3. Требования к персоналу (пользователю)

Специальных требований к персоналу данная программа не имеет.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. Выполнение программы

3.1. Установка программы

3.1.1. Скачивание программы

Для установки программы пользователь должен получить ее исходный код. Это можно сделать через загрузку из gitHub репозиторий или иным способом.

Команда для консоли для скачивания `git clone https://github.com/pipitochka/o7rc.git`

3.1.2. Сборка программы

Программа может быть собрана двумя способами: через Docker-контейнер или путем локальной сборки из исходных кодов.

1. Использование Docker (рекомендуемый)

1. Соберите Docker-образ, находясь в корневой директории проекта:

```
docker build -t o7rc .
```

2. Запустите интерактивный контейнер, примонтировав текущую директорию:

```
docker run -it --rm -v "$(pwd)":/work o7rc
```

3. Внутри контейнера выполните сборку:

```
mkdir build && cd build && cmake .. && cmake --build . --parallel
```

2. Локальная сборка

1. `mkdir build && cd build && cmake .. && cmake --build . --parallel`

В зависимости от пожеланий оператора при сборке могут быть использованы следующие cmake флаги:

Опция CMake	По умолчанию	Описание
USE_FLEX	ON	Использовать генерируемый Flex-лексер.
USE_BISON	ON	Использовать генерируемый Bison-парсер.
USE_HAND_TOKENIZER	OFF	Использовать рукописную реализацию лексера.
USE_HAND_PARSER	OFF	Использовать рукописную реализацию парсера (рекурсивный спуск).
USE_CODEGEN	ON	Включить прямую кодогенерацию ассемблера RISC-V (напрямую из AST в asm).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

USE_RARS	ON	Автоматически скачать RARS/OBNC при сборке и включить E2E-тесты.
USE_DEBUG	OFF	Включить отладочный вывод токенов в консоль.

Требуется хотя, чтобы был подключен хотя бы один из парсеров и токенайзеров для успешного завершения сборки. При отсутствии изменений флагов, их стандартные значения написаны выше.

3.1.3. Запуск программы

Запуск осуществляется как у стандартного CLI бинарного приложения

```
./o7rc input.obr -o output.asm
```

Доступные флаги CLI

Флаг / Параметр	Описание
-o <file>	Путь к выходному .asm файлу.
--tokenizer <flex hand>	Выбор лексера.
--parser <bison hand>	Выбор парсера.
--ir	Генерация кода через промежуточное представление (IR) вместо прямой трансляции AST → asm.
--opt	Включить проходы оптимизации (автоматически подразумевает использование --ir).
--dump-ir	Напечатать IR в стандартный поток ошибок stderr (до и после оптимизаций).
--dump-ir-passes	Напечатать состояние IR после каждого завершенного прохода оптимизации.
--no-sema	Отключить этап семантического анализа (проверки типов, связывания имен).
--dump-dot <file>	Экспортировать граф потока управления в формат DOT.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

При попытке использовать модуль, который не был скопирован, программа использует собранный. Если ни одна реализация модуля не была собрана, программа завершится с кодом 1 и соответствующим сообщением.

При нехватке информации о входном и выходном файлах, программа попросит ввести их при запуске

Ниже приведены типовые сценарии запуска кросс-компилятора, демонстрирующие комбинирование различных флагов командной строки в зависимости от задач разработчика.

Прямая трансляция исходного кода в ассемблер на основе обхода AST. Выходной файл будет иметь название `program.asm`.

2. Компиляция с включенными оптимизациями

Программа сначала переводится во внутреннее промежуточное представление, после чего к ней применяются проходы оптимизации. Выходной файл будет иметь название `program.asm`.

3. Компиляция с включенными оптимизациями

Если `main.obr` импортирует другие модули, необходимо указать директории для их поиска с помощью флага `-M`.

4. Выбор конкретной реализации лексера и парсера

Если `main.obr` импортирует другие модули, необходимо указать директории для их поиска с помощью флага `-M`.

5. Глубокая отладка процесса оптимизации

В данном режиме в стандартный поток ошибок (stderr) будет выводиться состояние IR-кода после каждого примененного прохода оптимизации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```
./o7rc program.obr -o program.asm --ir --opt --dump-ir-passes
```

6. Отключение семантического анализа

```
./o7rc program.obr -o program.asm --no-sema
```

Все данные флаги можно использовать одновременно.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. СООБЩЕНИЯ ОПЕРАТОРУ

4.1. Тексты сообщений оператору

При успешной работе программа завершится с кодом 0. В стандартный поток ввода-вывода будут печататься следующие сообщения в процессе работы:

Сообщение	Значение
Enter path to source file: Enter path to output file:	Интерактивный запрос пути к файлу, если он не был передан через аргументы командной строки.
Parsed module: <name>	Уведомление об успешном завершении лексического и синтаксического анализа (построении AST) модуля.
CFG exported: <path>	Уведомление об успешном экспорте графа потока управления (появляется только при использовании флага --dump-dot).
Generated: <path> Generated (IR): <path>	Уведомление об успешной генерации целевого ассемблерного кода и завершении работы программы.

В случае неуспешного выполнения программа завершится с кодом 1 и соответствующим сообщением в потоке ошибок stderr.

Текст сообщения	Причина	Действия оператора
No path provided	При интерактивном запросе пути к файлу оператор ввел пустую строку или прервал ввод.	Перезапустить программу. Рекомендуется передавать пути через аргументы (например, ./o7rc in.obr -o out.asm).
Cannot open file: <path> Cannot open output file: <path>	Система не может найти файл по указанному пути, либо у оператора отсутствуют права на чтение/запись.	Проверить корректность путей и отсутствие опечаток. Убедиться, что целевая директория доступна для записи.
No tokenizer built/ selected	Программа была собрана (CMake) без включения компо-	Пересобрать проект из исходных кодов с корректными флагами CMake

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

No parser built/ selected	ентов лексического или син- таксического анализатора.	(включить как минимум один лек- сер и парсер).
Parsing failed	В исходном коде на Обе- рон-7 обнаружены синтаксиче- ские ошибки.	Изучить предшествующие логи пар- сера. Открыть код и исправить опе- чатки.
Failed to load imported modules	Компилятор встретил IMPORT, но не смог найти или разобрать запрашиваемый внешний мо- дуль.	Убедиться, что файл модуля суще- ствует. Проверить, что путь к его ди- ректории передан флагом -M <dir>.
sema: <ошибка> sema [<module>]: <ошибка>	Семантический анализатор вы- явил логическую ошибку (необъявленная переменная, несовпадение типов).	Проанализировать текст <ошибка>. Внести исправления в типы, пере- менные или вызовы процедур в ис- ходном коде.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. ТЕСТИРОВАНИЕ

5.1. Юнит-тесты

Для юнит-тестов использована библиотека ctest

Запуск `ctest -E e2e --output-on-failure` или `./o7rc_tests`

Тесты запускаются для всех скомпилированных компонентов.

Юнит тесты расположены в следующих директориях и тестируют соответствующие компоненты.

Категория	Файлы	Описание
Лексер	tests/tokenizer/test_tokenizer_*.cpp	Токенизация, грамматика, сниппеты.
Парсер	tests/parser/test_parser_*.cpp	Структура AST, выражения, типы, операторы.
Sema	tests/sema/test_sema.cpp	27 тестов: позитивные + негативные.
IR Builder	tests/ir/test_ir_builder.cpp	Построение IR: модули, переменные, CFG.
IR Passes	tests/ir/test_ir_passes.cpp	ConstantFolding, CopyPropagation, DCE, PassManager.
IR Memory	tests/ir/test_ir_memory.cpp	RECORD, POINTER TO, NEW, доступ к полям, вложенные записи.

5.2. E2E-тесты

Требуется Java, libgc-dev и сборка с `-DUSE_RARS=ON`:

`cd build ctest -L e2e --output-on-failure`

Если для какого-то модуля собраны обе конфигурации, то e2e тесты будут проходить на обоих т.е. при полной сборке со всеми флагами ON каждый тест будет запускаться 8 раз (2 парсера, 2 токеназера, 2 кодогенератора)

Каждый запуск e2e теста работает по следующему сценарию

1. Компилирует .obg через OBNC → эталонный вывод

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. Компилирует .obr → .asm через o7rc → запускает в RARS → фактический вывод
3. Сравнивает выводы

Все тестовые программы:

Категория	Программы
Базовые	Hello, Arith, Assign, Negate, ConstTest
Управление	IfElse, NestedIf, CaseStmt, WhileLoop, RepeatTest, ForLoop, NestedLoop
Логика	BoolLogic, EvenOdd
Массивы	ArraySum, ArrayReverse, BubbleSort
Процедуры	Factorial, Fibonacci, MultiProc, Scope, MutualCall, RecSum
Записи / указатели	Records, RecordProc, LinkedList
Встроенные	IncDec, AbsOdd, DivMod
Алгоритмы	GCD, Power, Primes, SumDigits, Collatz
С входными данными	ReadSum, ReadMax, ReadFact, ReadGCD, ReadPower, ReadArray, ReadClassify
Модули	UseMath (импорт Math)

Тесты с входными данными — для программ, использующих In.Int, рядом с .obr файлом лежит .in файл с входными данными. Скрипт run_test.sh автоматически подаёт его на stdin как для OBNC, так и для RARS.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.04.01 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]